

CHAPTER 15

15. 실전 종합 — 게시판 API 한 앱으로

Node.js · Express · NestJS 강의 · 2026

지금까지 인증·ORM·캐싱을 따로따로 배웠다. 13장에서 JWT로 로그인을 만들었고, 12장에서 Prisma로 관계형 데이터를 다뤘고, 14장에서 Redis로 캐시를 붙였다. 각각은 독립된 예제였다. 실무에서는 이 셋이 한 앱 안에서 동시에 돌아간다. 글을 쓰려면 로그인해야 하고(인증), 글과 댓글은 데이터베이스에 관계로 저장되며(ORM), 자주 읽히는 글 상세는 캐시에서 빠르게 내려간다(캐싱).

이 장은 그 통합을 직접 만든다. 작은 게시판 API다. 회원가입·로그인이 있고, 로그인한 사용자만 글과 댓글을 쓸 수 있으며, 목록은 페이지 단위로 끊어 검색되고, 글 상세는 Redis에 캐시된다. 새 개념을 배우기보다 이미 배운 조각들이 어떻게 맞물리는지를 보는 장이다. 그래서 이 장의 핵심 질문은 하나다. "글을 쓸 때 작성자는 어디서 오는가?" 답은 요청 본문이 아니라 토큰이다. 그 한 줄에 인증·보안·데이터 설계가 모두 걸려 있다.

이 자료는 **개념 예습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

이 장에서 배우는 것

- 인증·사용자·글·댓글·DB 연결, 다섯 기능을 하나의 NestJS 앱으로 조립하는 구조
- 인증 가드로 쓰기 라우트를 막고, 작성자(`authorId`)를 **토큰에서** 가져오는 보안 패턴
- Prisma로 User · Post · Comment의 1:N 관계를 다루고, 응답에서 비밀번호를 빼는 법
- 목록의 페이지네이션·검색, 글 상세의 `cache-aside`와 댓글·삭제 시 캐시 무효화
- Swagger로 보호 라우트까지 포함한 API 문서를 자동 생성하는 법

기능은 다섯 갈래로 나뉜다. 각자 역할이 분명하고, 서로 필요한 만큼만 의존한다.

기능	핵심 역할	통합 포인트
DB 연결	커넥션 한 곳, 전역 제공	모두가 공유
사용자	사용자 조회, 비밀번호 제외 <code>select</code>	DB
인증	회원가입·로그인·JWT 발급, 가드·전략	사용자, JWT
글	글 CRUD, 페이지네이션·검색, 캐시	DB, 캐시, 인증 가드
댓글	댓글 작성·조회, 글 캐시 무효화	DB, 캐시, 인증 가드

핵심 개념 — 세 기술이 한 요청 안에서 만나는 자리

"글 하나를 쓴다"는 평범한 요청을 따라가 보면, 앞 장들에서 배운 것이 차례로 등장한다.

1. **인증(13장)** — 요청 헤더의 `Authorization: Bearer <토큰>` 을 인증 가드가 검사한다. 토큰이 없거나 위조됐으면 라우트에 닿기 전에 401로 막힌다. 통과하면 토큰에서 푼 사용자 정보가 `req.user` 에 담긴다.
2. **데이터(12장)** — 컨트롤러는 본문(제목·내용)과 `req.user.id` 를 합쳐 글 서비스에 넘기고, 서비스는 Prisma로 `Post` 한 행을 만든다. `authorId` 는 이때 토큰에서 온 값으로 박힌다.
3. **캐싱(14장)** — 새 글이니 아직 캐시와는 무관하다. 하지만 누군가 이 글을 읽으면 상세가 Redis에 적재되고, 댓글이 달리거나 글이 지워지면 그 캐시는 즉시 버려진다.

즉 세 기술은 **요청 파이프라인의 서로 다른 칸**을 맡는다. 가드는 입구를 지키고, ORM은 데이터를 다루고, 캐시는 읽기를 빠르게 한다. 한 칸씩 떼어 보면 13·12·14장에서 본 그대로다. 이 장에서 새로 다루는 건 그 칸들을 잇는 배선이다.

왜 작성자를 토큰에서 가져오나 — 글쓰기 요청 본문에 `authorId` 를 받으면, 클라이언트가 `authorId: 999` 라고 보내 남의 이름으로 글을 쓸 수 있다. 위조다. 그래서 작성자는 **클라이언트가 보내는 값이 아니라, 서버가 토큰을 검증해 알아낸 값**이어야 한다. 글 생성 DTO에 `authorId` 필드가 아예 없는 이유가 이것이다. 이 한 가지 결정에 게시판의 보안이 달려 있다.

목록은 왜 캐시하지 않나 — 글 상세(`GET /posts/:id`)는 캐시하지만, 목록(`GET /posts`)은 캐시하지 않는다. 목록은 페이지·검색어 조합마다 결과가 다르고, 글이 하나만 추가돼도 바로 바뀐다. 캐시 적중률은 낮고 무효화 비용은 큰 자리라, 굳이 캐시하지 않는 편이 단순하고 정확하다. 캐시는 **자주 읽히고 잘 안 바뀌는** 데이터에 쓴다.

준비

PostgreSQL과 Redis가 모두 필요하다. 인증·데이터는 Postgres에, 캐시는 Redis에 들어간다.

```
# 빈 DB 생성
createdb nest_board

# Redis 설치 & 실행 (macOS / Homebrew)
brew install redis
brew services start redis
```

그다음 의존성 설치와 환경변수, 마이그레이션이다.

```
npm install
cp .env.example .env
npx prisma migrate dev--name init
```

`.env` 는 네 가지를 채운다. `DATABASE_URL` 의 `USER:PASSWORD` 는 본인 로컬 Postgres 계정으로 바꾼다.

```

DATABASE_URL="postgresql://USER:PASSWORD@localhost:5432/nest_board?schema=public"
JWT_SECRET="dev-secret-change-me"
REDIS_HOST=localhost
REDIS_PORT=6379
PORT=3000

```

`npx prisma migrate dev` 는 Prisma 스키마를 읽어 `User · Post · Comment` 테이블을 만들고, Prisma 클라이언트도 함께 생성한다. 준비가 끝나면 개발 서버를 띄운다.

```

npm run start:dev      # → http://localhost:3000 (Swagger 문서: /docs)

```

서버가 뜨면 브라우저로 `http://localhost:3000/docs` 에 접속해 Swagger 문서부터 확인하자. 아래에서 설명할 라우트가 모두 거기 정리돼 있고, 자물쇠 표시가 붙은 라우트가 로그인을 요구하는 보호 라우트다.

앱 조립 — 루트 모듈이 다섯 기능을 합친다

먼저 전체 구조다. 루트 모듈은 직접 일하지 않는다. 다섯 기능 모듈과 전역 캐시를 한자리에 모아 NestJS에 등록할 뿐이다.

```

@Module({
  imports: [
    // 14장의 Redis 캐시 - 전역
    CacheModule.registerAsync({
      isGlobal: true,
      useFactory: async () => ({
        store: await redisStore({
          socket: {
            host: process.env.REDIS_HOST ?? "localhost",
            port: Number(process.env.REDIS_PORT ?? 6379),
          },
          ttl: 30_000,
        }),
      }),
    }),
    PrismaModule, // @Global
    UsersModule,
    AuthModule, // 13장의 인증 (JWT 전략 등록)
    PostsModule,
    CommentsModule,
  ],
})
export class AppModule {}

```

- `CacheModule.registerAsync({ isGlobal: true })` — Redis 캐시를 전역으로 등록한다. `isGlobal` 덕분에 글·댓글 서비스가 따로 import하지 않아도 `CACHE_MANAGER` 를 주입받는다. `ttl: 30_000` 은 기본 만료 시간 30초(밀리초 단위)다.
- DB 연결 모듈도 `@Global` 이라 마찬가지로(잠시 뒤에 본다). DB 연결과 캐시처럼 앱 전체가 공유하는 인프라는 전역으로 한 번만 등록하는 게 관례다.
- 나머지 사용자·인증·글·댓글은 각자 컨트롤러와 서비스를 들고 있는 기능 모듈이다. 등록 순서 자체에는 의미가 없지만, 의존 관계상 인증이 사용자를 쓰므로 둘은 늘 함께 있다.

앱 부트스트랩 진입점은 이 루트 모듈을 띄우고 두 가지 전역 설정을 건다.

```
import "dotenv/config"; // .env → process.env (JWT_SECRET·REDIS_* 를 모듈보다 먼저 로드)
//...
async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  // 전역 검증 (13~15와 동일한 계층)
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));

  const config = new DocumentBuilder()
    .setTitle("게시판 API")
    .setDescription("NestJS + Prisma 실전 종합 - users / posts / comments")
    .setVersion("1.0")
    .addBearerAuth() // 보호 라우트 테스트용 토큰 입력
    .build();
  SwaggerModule.setup("docs", app, SwaggerModule.createDocument(app, config));
  //...
}
```

- 맨 윗줄 `import "dotenv/config"` 가 가장 먼저 와야 한다. `JWT_SECRET` 이나 `REDIS_HOST` 같은 값이 `process.env` 에 채워지는 시점은 이 import가 실행될 때다. 만약 이게 뒤에 오면, 비밀키 상수처럼 모듈을 불러올 때 `process.env.JWT_SECRET` 을 읽는 코드가 `undefined` 를 보게 된다.
- `ValidationPipe` 의 `whitelist: true` 는 DTO에 정의되지 않은 필드를 자동으로 걸러 낸다. 클라이언트가 `authorId` 를 몰래 끼워 보내도 DTO에 없으니 버려진다. 작성자 위조를 막는 또 한 겹의 방어선이다.
- `transform: true` 는 들어온 값을 DTO 타입에 맞춰 변환한다. 뒤에서 볼 쿼리스트링 `?page=2` 의 `"2"` 를 숫자 2로 바꾸는 일이 여기서 일어난다.
- `addBearerAuth()` — Swagger 문서에 토큰 입력 칸을 더한다. `/docs` 에서 우측 상단 **Authorize** 버튼으로 토큰을 한 번 넣어 두면, 보호 라우트도 문서 화면에서 바로 테스트할 수 있다.

DB 연결 — 커넥션 한 곳

데이터 계층의 토대다. DB 연결 서비스는 Prisma 클라이언트를 NestJS 생명주기에 연결한 얇은 래퍼다.

```

@Injectable()
export class PrismaService
  extends PrismaClient
  implements OnModuleInit, OnModuleDestroy
{
  async onModuleInit() {
    await this.$connect();
  }

  async onModuleDestroy() {
    await this.$disconnect();
  }
}

```

`PrismaClient` 를 상속하므로 `this.user`, `this.post`, `this.comment` 같은 모델 API를 그대로 쓴다. 거기에 NestJS의 생명주기 혹은 두 개를 더했다. 모듈이 뜰 때 `$connect()` 로 DB에 붙고, 앱이 내려갈 때 `$disconnect()` 로 연결을 닫는다. 커넥션을 한 번만 열고 앱과 함께 정리하는, 가장 표준적인 형태다.

```

@Global()
@Module({
  providers: [PrismaService],
  exports: [PrismaService],
})
export class PrismaModule {}

```

`@Global()` 이 핵심이다. 이게 있어 어느 모듈이든 DB 연결 모듈을 import하지 않고도 생성자에서 DB 연결 서비스를 주입받는다. DB는 앱 전체가 공유하는 자원이니, 모듈마다 import 구문을 반복하지 않으려는 의도다.

데이터 모델은 Prisma 스키마에 정의돼 있다. 세 모델이 1:N 관계로 엮인다.

```

model User {
  id      Int      @id @default(autoincrement())
  email   String   @unique
  password String // bcrypt 해시 (인증용)
  name    String
  posts   Post[]
  comments Comment[]
  createdAt DateTime @default(now())
}

model Post {
  id      Int      @id @default(autoincrement())
  title   String
  content String
  author  User      @relation(fields: [authorId], references: [id], onDelete: Cascade)
  authorId Int
  comments Comment[]
  createdAt DateTime @default(now())
}

model Comment {
  id      Int      @id @default(autoincrement())
  content String
  post    Post      @relation(fields: [postId], references: [id], onDelete: Cascade)
  postId  Int
  author  User      @relation(fields: [authorId], references: [id], onDelete: Cascade)
  authorId Int
  createdAt DateTime @default(now())
}

```

- `User 1:N Post`, `Post 1:N Comment`, `User 1:N Comment` — 한 사용자가 여러 글과 댓글을 쓰고, 한 글에 여러 댓글이 달린다. 12장의 단순 관계에 `Comment` 를 더해 한 단계 깊어졌다.
- `password` 는 평문이 아니라 **bcrypt 해시**가 들어간다(주석 그대로). 모델에는 그냥 `String` 이지만, 저장 직전에 해시하는 책임은 인증 서비스에 있다.
- `onDelete: Cascade` 가 세 관계에 모두 걸려 있다. 사용자를 지우면 그가 쓴 글·댓글이 같이 지워지고, 글을 지우면 그 글의 댓글도 같이 지워진다. 고아 데이터가 남지 않게 한다. 뒤의 글 삭제가 댓글을 따로 지우지 않아도 되는 이유가 이것이다.

인증 — 회원가입·로그인·JWT

인증은 13장에서 만든 것을 거의 그대로 가져왔다. 흐름은 둘이다. **회원가입**은 비밀번호를 해시해 저장하고, **로그인**은 비밀번호를 검증해 토큰을 발급한다.

회원가입 — 비밀번호 해시

```
async register(dto: RegisterDto) {
  const exists = await this.userService.findByEmail(dto.email);
  if (exists) throw new ConflictException("이미 가입된 이메일입니다.");

  const hashed = await bcrypt.hash(dto.password, 10);
  const user = await this.userService.create({
    email: dto.email,
    name: dto.name,
    password: hashed,
  });
  const { password, ...result } = user;
  return result;
}
```

- 먼저 이메일 중복을 확인한다. 이미 있으면 **409 Conflict**다. "요청 자체는 멀쩡한데 현재 상태와 충돌한다"는 뜻으로, 잘못된 입력(400)과 구분된다.
- `bcrypt.hash(dto.password, 10)` — 평문 비밀번호를 해시한다. `10` 은 해시 강도(salt rounds)다. DB에 평문을 저장하지 않는 건 인증의 기본 원칙이다.
- 마지막 `const { password, ...result } = user` 에 주목하자. 저장한 결과에서 `password` 만 떼어 내고 나머지 (`result`)만 응답한다. 가입 직후 응답에도 해시가 새 나가지 않게 막는다.

로그인 — 검증 후 토큰 발급

```
async login(email: string, password: string) {
  const user = await this.userService.findByEmail(email);
  if (!user || !(await bcrypt.compare(password, user.password))) {
    throw new UnauthorizedException(
      "이메일 또는 비밀번호가 올바르지 않습니다."
    );
  }
  const payload = { sub: user.id, email: user.email };
  return { access_token: this.jwtService.sign(payload) };
}
```

- `bcrypt.compare(평문, 해시)` — 들어온 평문을 저장된 해시와 비교한다. 해시는 되돌릴 수 없으니 "평문을 다시 해시해 같은지 보는" 방식이다.
- 사용자가 없거나 비밀번호가 틀리면 **401 Unauthorized**다. 이때 "이메일이 없다"와 "비밀번호가 틀리다"를 구분하지 않고 같은 메시지를 쓴다. 어느 이메일이 가입돼 있는지 공격자에게 흘리지 않으려는 작은 보안 습관이다.
- `payload = { sub: user.id, email }` — 토큰에 담을 정보다. `sub` (subject)에 사용자 id를 넣는 건 JWT 관례다. 이 id가 나중에 글·댓글의 `authorId` 가 된다.

- `jwtService.sign(payload)` — 페이로드에 서명해 토큰 문자열을 만든다. 서명 키와 만료는 인증 모듈의 `JwtModule.register` 에서 비밀키 상수와 `expiresIn: "1h"` 로 정해 둔다.

가드와 전략 — 토큰을 검증해 `req.user` 로

토큰을 발급했으니, 다음은 그 토큰을 검증하는 쪽이다. 둘이 짝을 이룬다. **JWT 전략**이 토큰을 풀어 검증하고, **인증 가드**가 그 전략을 라우트 앞에 세운다.

```
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: jwtConstants.secret,
    });
  }

  // 반환값이 req.user 가 된다. payload = { sub, email }
  async validate(payload: any) {
    return { id: payload.sub, email: payload.email };
  }
}
```

- `fromAuthHeaderAsBearerToken()` — 토큰을 `Authorization: Bearer <토큰>` 헤더에서 뽑는다. 로그인 때 발급한 토큰을 클라이언트가 이 헤더에 실어 보낸다.
- `ignoreExpiration: false` — 만료된 토큰은 거부한다. `secretOrKey` 는 발급 때와 같은 **비밀키**여야 한다. 그래서 비밀키 상수가 단일 출처로 키를 들고 있다.
- `validate(payload)` 의 반환값이 곧 `req.user` 다. 여기서 `{ id: payload.sub, email }` 을 돌려주므로, 컨트롤러에서 `req.user.id` 가 토큰의 `sub` (=사용자 id)가 된다. 이 한 줄이 "작성자를 토큰에서 가져온다"의 출발점이다.

```
@Injectable()
export class JwtAuthGuard extends AuthGuard("jwt") {}
```

가드 자체는 단 한 줄이다. `AuthGuard("jwt")` 는 위의 JWT 전략을 라우트 앞에 끼워 넣는다. 컨트롤러에서 `@UseGuards(JwtAuthGuard)` 를 붙인 라우트는, 핸들러가 실행되기 전에 토큰 검증을 통과해야 한다. 통과하면 `req.user` 가 채워지고, 실패하면 401로 막힌다.

사용자 — 조회 전용, 비밀번호 제외

사용자 생성은 `/auth/register` 가 맞는다(비밀번호 해시가 필요하니까). 그래서 사용자 컨트롤러에는 조회 라우트만 있다. 핵심은 응답에서 비밀번호를 빼는 `select` 다.

```
@Injectable()
export class UsersService {
  constructor(private readonly prisma: PrismaService) {}

  create(data: { email: string; name: string; password: string }) {
    return this.prisma.user.create({ data });
  }

  // 로그인 검증용 - password(해시) 포함해서 가져온다.
  findByEmail(email: string) {
    return this.prisma.user.findUnique({ where: { email } });
  }

  // 공개 조회 - password는 select에서 제외해 노출되지 않게 한다.
  findAll() {
    return this.prisma.user.findMany({
      select: {
        id: true,
        email: true,
        name: true,
        createdAt: true,
        _count: { select: { posts: true, comments: true } },
      },
    });
  }
  //...
}
```

여기서 메서드 둘의 차이가 중요하다.

- `findByEmail` 은 `select` 없이 전부 가져온다. 로그인 검증에 쓰이는 내부용이라 `password` 해시가 필요하기 때문이다. 로그인 처리가 이 해시를 `bcrypt.compare` 에 쓴다.
- `findAll` · `findOne` 은 `select` 로 `id` · `email` · `name` · `createdAt` 만 골라 온다. 공개 응답용이라 `password` 를 의도적으로 뺀다. Prisma의 `select` 는 "고른 필드만" 가져오므로, `password` 를 안 적으면 애초에 조회조차 안 된다. 응답을 가공해 지우는 게 아니라 처음부터 안 가져오는 방식이라 더 안전하다.
- `_count: { select: { posts: true, comments: true } }` — 관계의 개수만 센다. 글·댓글 본문을 다 끌어오지 않고 "몇 개 썼는지"만 숫자로 더한다.

같은 `User` 모델을 쓰지만 용도에 따라 가져오는 필드가 다르다는 점이 이 설계의 포인트다. 비밀번호가 새 나가는 사고는 대개 "전부 가져와 그대로 응답"에서 난다. `select` 로 출구를 좁혀 두면 그 실수가 원천 봉쇄된다.

글 — 글쓰기 보호 · 페이지네이션 · 캐시

게시판의 본체다. 인증·데이터·캐시가 한 서비스에서 모두 만난다. 컨트롤러부터 보자.

```
@ApiTags("posts")
@Controller("posts")
export class PostsController {
  constructor(private readonly postsService: PostsService) {}

  // 글쓰기 — 로그인 필요. authorId는 토큰의 사용자로 자동 지정.
  @Post()
  @UseGuards(JwtAuthGuard)
  @ApiBearerAuth()
  create(@Body() dto: CreatePostDto, @Request() req) {
    return this.postsService.create(dto, req.user.id);
  }

  @Get() // 공개 — GET /posts?page=1&limit=10&search=키워드
  findAll(@Query() query: QueryPostDto) {
    return this.postsService.findAll(query);
  }

  @Get(":id") // 공개 — 단건(댓글 포함, 캐시됨)
  findOne(@Param("id") id: string) {
    return this.postsService.findOne(+id);
  }

  @Delete(":id") // 삭제 — 로그인 필요
  @UseGuards(JwtAuthGuard)
  @ApiBearerAuth()
  remove(@Param("id") id: string) {
    return this.postsService.remove(+id);
  }
}
```

라우트가 둘로 갈린다. 읽기(`findAll` · `findOne`)는 누구나 접근하는 공개 라우트고, 쓰기(`create` · `remove`)는 `@UseGuards(JwtAuthGuard)` 로 로그인을 요구한다. 게시판의 자연스러운 권한 모델이다 — 글은 누구나 읽지만 아무나 쓰지는 못한다.

`create` 의 마지막 인자에 주목하자. `this.postsService.create(dto, req.user.id)` 다. 본문(`dto`)에는 제목·내용만 있고, 작성자는 `req.user.id` 로 따로 넘긴다. 이 `req.user` 는 위에서 본 JWT 전략의 `validate` 가 채운 값이다. 토큰을 거쳐 온 사용자 id이니 위조할 수 없다.

글 생성 DTO를 보면 의도가 분명하다.

```
// authorId는 body로 받지 않는다 - 로그인한 사용자(JWT)로부터 가져온다.
export class CreatePostDto {
  @ApiProperty({ example: "첫 글" })
  @IsString()
  @MinLength(1)
  title: string;

  @ApiProperty({ example: "안녕하세요, 본문입니다." })
  @IsString()
  content: string;
}
```

`authorId` 필드가 없다. DTO에 없으니 `whitelist: true` 설정과 맞물려, 클라이언트가 `authorId` 를 보내도 무시된다. 작성자를 본문으로 받을 길 자체를 막아 뒀다.

글 생성 — 작성자는 인자로 받는다

```
// authorId는 컨트롤러가 JWT(req.user.id)에서 넘겨준다.
async create(dto: CreatePostDto, authorId: number) {
  return this.prisma.post.create({ data: { ..dto, authorId } });
}
```

서비스는 `dto` (제목·내용)와 컨트롤러가 넘긴 `authorId` 를 합쳐 한 행을 만든다. `{ ..dto, authorId }` 로 펼치면 `{ title, content, authorId }` 가 되어 `Post` 모델에 그대로 들어간다. 작성자의 출처가 본문이 아니라 함수 인자라는 점이 이 코드의 보안 핵심이다.

목록 — 페이지네이션과 검색

```

async findAll(query: QueryPostDto) {
  const { page, limit, search } = query;
  const where = search
    ? {
      OR: [
        { title: { contains: search, mode: "insensitive" as const } },
        { content: { contains: search, mode: "insensitive" as const } },
      ],
    }
    : {};

  const [items, total] = await Promise.all([
    this.prisma.post.findMany({
      where,
      skip: (page - 1) * limit,
      take: limit,
      orderBy: { id: "desc" },
      include: {
        author: { select: { id: true, name: true } },
        _count: { select: { comments: true } },
      },
    }),
    this.prisma.post.count({ where }),
  ]);

  return { items, total, page, limit, totalPages: Math.ceil(total / limit) };
}

```

- `where` — 검색어가 있으면 제목 또는 내용에 그 문자열이 포함된 글만 거른다. `OR` 배열로 두 조건을 묶고, `mode: "insensitive"` 로 대소문자를 가리지 않는다.
- `skip` / `take` — 페이지네이션의 핵심이다. `skip: (page - 1) * limit` 로 앞 페이지를 건너뛰고, `take: limit` 로 한 페이지 분량만 가져온다. `page=2, limit=10` 이면 11번째부터 10개다.
- `Promise.all([.])` — **현재 페이지 데이터**(`findMany`)와 **전체 개수**(`count`)를 동시에 조회한다. 둘은 서로 의존하지 않으니 순차로 기다릴 이유가 없다. 같은 `where` 를 양쪽에 써서 "검색 결과의 전체 개수"를 정확히 센다.
- 응답에 `total` 과 `totalPages` 를 함께 담는다. 클라이언트가 "전체 몇 개 중 몇 페이지인지"를 알아야 페이지 UI를 그릴 수 있어서다.
- `include` 로 작성자 이름과 댓글 수를 곁들인다. 여기서도 `author` 는 `select` 로 `id · name` 만 — 목록에 작성자의 이메일까지 노출할 이유는 없다.

쿼리스트링을 받는 목록 조회 DTO는 숫자 변환과 기본값을 맡는다.

```
// 쿼리스트링은 항상 문자열이라 @Type(() => Number)로 숫자 변환이 필요하다.
export class QueryPostDto {
  @ApiPropertyOptional({ default: 1, description: "페이지 번호(1부터)" })
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  page: number = 1;

  @ApiPropertyOptional({ default: 10, description: "페이지 크기" })
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  limit: number = 10;

  @ApiPropertyOptional({ description: "제목/내용 검색어" })
  @IsOptional()
  @IsString()
  search?: string;
}
```

쿼리스트링 `?page=2` 의 `"2"` 는 문자열이다. `@Type(() => Number)` 가 이를 숫자로 바꾸고(전역 설정의 `transform: true` 와 함께 작동한다), `@IsInt()` · `@Min(1)` 이 1 이상의 정수인지 검증한다. `page: number = 1` 처럼 기본값을 줘서, 쿼리를 생략하면 1페이지·10개가 자동 적용된다.

글 상세 — cache-aside

이 장에서 가장 눈여겨볼 메서드다. 14장의 `cache-aside` 패턴이 여기 들어온다.

```
// 단건 상세(댓글 포함) - cache-aside. 댓글이 달리면 댓글 쪽에서 무효화한다.
async findOne(id: number) {
  const key = `post:${id}`;
  const cached = await this.cache.get(key);
  if (cached) return cached;

  const post = await this.prisma.post.findUnique({
    where: { id },
    include: {
      author: { select: { id: true, name: true } },
      comments: { include: { author: { select: { id: true, name: true } } } },
    },
  });
  if (!post) throw new NotFoundException(`게시글 ${id}를 찾을 수 없습니다.`);
  await this.cache.set(key, post, 30_000);
  return post;
}
```

cache-aside는 "캐시를 곁에 두고 직접 행기는" 방식이다. 흐름은 셋이다.

1. 캐시 먼저 조회 — `cache.get('post:1')` . 있으면(`cached`) 그대로 돌려주고 DB는 건드리지 않는다. 캐시 히트다.
2. 없으면 DB 조회 — 캐시가 비었으면 Prisma로 글을 가져온다. 작성자·댓글·댓글 작성자까지 한 번에 `include` 해서, 상세 화면에 필요한 걸 다 채운다.
3. 캐시에 적재 — 가져온 결과를 `cache.set(key, post, 30_000)` 으로 30초간 저장한다. 다음 같은 요청은 1번에서 끝난다.

키는 `post:${id}` 다. 글마다 고유한 키를 써야 캐시가 서로 섞이지 않는다. 이 키 규칙은 댓글 쪽에서도 똑같이 쓰이니 기억해 두자.

왜 곁들인 댓글까지 캐시에 담나 — 상세 응답에는 글 본문뿐 아니라 댓글 목록도 포함된다. 통째로 캐시하면 읽기는 빠르지만, 댓글이 하나 달리는 순간 캐시는 옛날 데이터(**stale**)가 된다. 그래서 무효화가 반드시 짝으로 따라온다. 다음 두 곳에서 이 캐시를 버린다.

글 삭제 — 캐시도 함께 정리

```
async remove(id: number) {
  try {
    await this.prisma.post.delete({ where: { id } });
  } catch {
    throw new NotFoundException(`게시글 ${id}를 찾을 수 없습니다.`);
  }
  await this.cache.del(`post:${id}`); // 캐시도 정리
}
```

- Prisma의 `delete` 는 대상이 없으면 예외를 던진다. 그걸 `try/catch` 로 잡아 **404**로 바꾼다. 없는 글을 지우려는 요청에 정확한 상태코드를 돌려주는 처리다.
- 삭제 뒤 `cache.del('post:1')` 로 그 글의 캐시도 버린다. 안 그러면 DB에서는 지워졌는데 캐시에는 30초간 남아, `GET /posts/1` 이 이미 없는 글을 돌려주는 모순이 생긴다.
- 글이 지워지면 그 글의 댓글도 `onDelete: Cascade` 로 DB에서 함께 사라진다. 댓글을 따로 지우는 코드가 없는 이유다.

댓글 — 댓글 작성과 캐시 무효화

댓글은 짧지만 두 가지를 분명히 보여 준다. 댓글도 로그인에 필요하다는 것, 그리고 댓글이 달리면 글 캐시를 무효화한다는 것.

```
// 댓글 작성 - 로그인 필요. 작성자는 토큰에서.
@Post()
@UseGuards(JwtAuthGuard)
@ApiBearerAuth()
create(@Body() dto: CreateCommentDto, @Request() req) {
  return this.commentsService.create(dto, req.user.id);
}

@Get() // 공개 - GET /comments?postId=1
findByPost(@Query("postId") postId: string) {
  return this.commentsService.findByPost(+postId);
}
```

글쓰기와 판박이다. 작성은 `JwtAuthGuard` 로 보호하고 작성자는 `req.user.id` 로 넘긴다. 댓글 생성 DTO에도 `authorId` 는 없고 `content` 와 `postId` 만 있다. "어느 글에 다는가"는 본문으로 받지만, "누가 다는가"는 토큰에서 가져온다.

서비스가 핵심이다.

```
// authorId는 컨트롤러가 JWT(req.user.id)에서 넘겨준다.
async create(dto: CreateCommentDto, authorId: number) {
  const post = await this.prisma.post.findUnique({
    where: { id: dto.postId },
  });
  if (!post) throw new BadRequestException(`게시글 ${dto.postId}가 없습니다.`);

  const comment = await this.prisma.comment.create({
    data: { content: dto.content, postId: dto.postId, authorId },
  });

  // 이 게시글 상세가 캐시돼 있으면 stale → 무효화 (글 상세와 동일 키 규칙)
  await this.cache.del(`post:${dto.postId}`);
  return comment;
}
```

- 먼저 대상 글이 있는지 확인한다. 없으면 **400 Bad Request**다 — 존재하지 않는 글에 댓글을 달려는 잘못된 요청이다.
- 댓글을 만들 때 `authorId` 는 인자로 받은 토큰의 사용자다. 글쓰기와 같은 보안 패턴이다.
- 마지막 줄이 이 모듈의 존재 이유다. `cache.del('post:${dto.postId}')` — 댓글이 달린 글의 캐시를 버린다. 키 규칙(`post:${id}`)이 글 상세 조회와 정확히 같아야 무효화가 맞아떨어진다. 다음에 누가 `GET /posts/1` 을 하면 캐시가 비었으니 DB에서 새로 읽고, 방금 단 댓글이 포함된 최신 상태가 다시 캐시된다.

여기가 글과 댓글이 캐시 키로 연결되는 지점이다. 글 쪽은 캐시를 채우고, 댓글 쪽은 같은 키로 그 캐시를 비운다. 키 문자열 하나가 두 기능의 약속이다. 그래서 `post:${id}` 형식을 한쪽에서 바꾸면 다른 쪽도 같이 바뀌어야 한다 — 안 그러면 무효화가 빗나가 stale 캐시가 남는다.

한 요청의 전체 흐름

"로그인한 사용자가 글에 댓글을 단다"를 처음부터 끝까지 따라가면, 다섯 기능이 어떻게 손을 맞추는지 한눈에 보인다.

```

POST /comments (헤더: Authorization: Bearer <토큰>, 본문: { content, postId })
├── [인증 가드] — JWT 전략이 토큰 검증 —▶ req.user = { id, email } (인증)
│                               실패 시 여기서 401
├── [전역 검증 파이프] — 댓글 생성 DTO 검증, authorId 같은 잉여 필드 제거
├── [댓글 컨트롤러] — commentsService.create(dto, req.user.id)
├── [댓글 서비스]
│   ├── 대상 글 존재 확인 (없으면 400) (DB)
│   ├── comment.create({ content, postId, authorId }) (DB)
│   └── cache.del('post:${postId}') — 글 상세 캐시 무효화 (캐시)
└── 201 + 생성된 댓글
  
```

가드(인증), 파이프(검증), 서비스(데이터), 캐시(무효화)가 한 줄에 늘어선다. 13·12·14장이 각각 맡았던 칸들이다.

정리 표 — 라우트와 권한

라우트	메서드	권한	핵심
/auth/register	POST	공개	비밀번호 해시 저장, 응답에서 제외
/auth/login	POST	공개	검증 후 JWT 발급
/auth/profile	GET	로그인	req.user 그대로 반환
/users · /users/:id	GET	공개	select 로 비밀번호 제외
/posts	GET	공개	페이지네이션 + 검색 (캐시 안 함)
/posts/:id	GET	공개	상세 + 댓글, cache-aside
/posts	POST	로그인	authorId 는 토큰에서
/posts/:id	DELETE	로그인	삭제 + 캐시 무효화
/comments	POST	로그인	authorId 는 토큰, 글 캐시 무효화
/comments?postId=	GET	공개	글별 댓글 목록

흔한 실수

- `authorId` 를 DTO나 본문으로 받는다. → 클라이언트가 남의 id를 보내 작성자를 위조한다. 작성자는 반드시 `req.user.id` 에서. DTO에는 `authorId` 필드를 두지 않는다.
- 댓글·삭제 후 캐시를 안 버린다. → DB는 바뀌었는데 `GET /posts/:id` 가 30초간 옛 데이터(댓글 빠진 글, 이미 지운 글)를 돌려준다. 쓰기마다 같은 키로 `cache.del` 을 짝지어야 한다.
- 캐시 키 규칙이 자리마다 다르다. → 글 쪽은 `post:1` 로 채우는데 댓글 쪽이 `posts:1` 로 지우면 무효화가 빚나간다. 키 형식은 한 가지로 통일한다.
- `select` 없이 사용자를 응답한다. → `password` 해시가 그대로 노출된다. 공개 조회는 항상 `select` 로 필요한 필드만.
- `import "dotenv/config"` 를 부트스트랩 진입점 뒤쪽에 둔다. → `JWT_SECRET` 이 모듈 로드 시점에 `undefined` 라 서명·검증 키가 어긋난다. 맨 위에 둔다.
- `whitelist: true` 를 빠뜨린다. → DTO에 없는 잉여 필드가 그대로 통과해, 작성자 위조 같은 입력이 새 들어온다.

직접 해보기

1. 본인 글만 삭제 — 지금 `DELETE /posts/:id` 는 로그인만 하면 남의 글도 지운다. 서비스에서 글의 `authorId` 와 `req.user.id` 를 비교해, 다르면 `ForbiddenException (403)`을 던지게 고쳐 보자. 컨트롤러에서 `req.user.id` 를 서비스로 넘기는 것부터 시작이다.
2. 댓글 수정·삭제 라우트 추가 — `PATCH /comments/:id` , `DELETE /comments/:id` 를 만들어 보자. 둘 다 로그인이 필요하고, 작업 뒤에는 해당 댓글이 속한 글의 캐시(`post:${postId}`)를 무효화해야 한다. 댓글에서 `postId` 를 먼저 조회해야 어떤 키를 지울지 알 수 있다.
3. 글 목록에도 짧은 캐시 — 자주 바뀌는 목록이라 캐시를 안 했지만, 1페이지·검색어 없는 가장 흔한 요청(`GET /posts`)만 5초쯤 캐시해 보자. 키에 `page · search` 를 넣어야 조합마다 섞이지 않는다. 그리고 새 글이 써지면 (`create`) 목록 캐시도 함께 무효화해야 한다.

정리

- 루트 모듈이 다섯 기능(DB 연결·사용자·인증·글·댓글)과 전역 캐시를 한 앱으로 조립한다. DB와 캐시는 `@Global` 로 한 번만 등록해 모두가 공유한다.
- 쓰기 라우트는 `JwtAuthGuard` 로 막고, 작성자(`authorId`)는 본문이 아니라 토큰에서 가져온다. JWT 전략의 `validate` 가 채운 `req.user.id` 가 그 출처다. DTO에 `authorId` 를 두지 않고 `whitelist` 로 잉여 필드를 거르는 것까지가 한 세트다.
- 비밀번호는 `bcrypt`로 해시해 저장하고, 공개 응답에서는 `select` 로 처음부터 가져오지 않는다.
- 목록은 페이지네이션·검색으로 끊어 내려 주고, 글 상세는 `cache-aside`로 캐시하되 댓글·삭제 시 같은 키로 무효화한다. 글과 댓글이 `post:${id}` 키 하나로 연결된다.

여기까지가 NestJS 과정의 끝이다. 10장에서 NestJS의 모듈·컨트롤러·서비스 구조를 익히고, 11·12장에서 TypeORM과 Prisma로 데이터베이스를 다루고, 13장에서 JWT 인증을, 14장에서 Redis 캐싱과 스케줄링을 붙였다. 흩어져 있던 그 조각들이 이 장에서 하나의 게시판 도메인으로 모였다. 인증이 입구를 막고, ORM이 관계형 데이터를 다루고, 캐시가 읽기를 빠르게 하는, 실무에서 마주칠 백엔드의 축소판이다. 앞으로 새 기능을 더할 때는 "이건 어느 칸에 들어가는가"를 먼저 따져 보자. 가드인지, 서비스인지, 캐시인지. 그 질문에 답할 수 있으면 이 과정이 목표한 만큼은 익힌 것이다.